AgenticStudio — Multi-Agent Job Platform Spec

Sep 27, 2026 · @VJ

Restructures AgenticStudio from a single-agent LangGraph loop into a Job-based multi-agent platform: a Job defines a team of .md-authored agent roles (e.g. QA Manager -> QA Lead -> Automation Engineer -> Test Executor), each with its own tasks, wired into a supervisor-orchestrated workflow that end users can trigger manually or on a schedule, and admins can watch execute node-by-node with full logs.

1. Core concept: Job -> Agents -> Tasks

The current codebase has one flat concept: a single planner/tool_executor/reflector loop per thread. It is restructured into three nested levels:

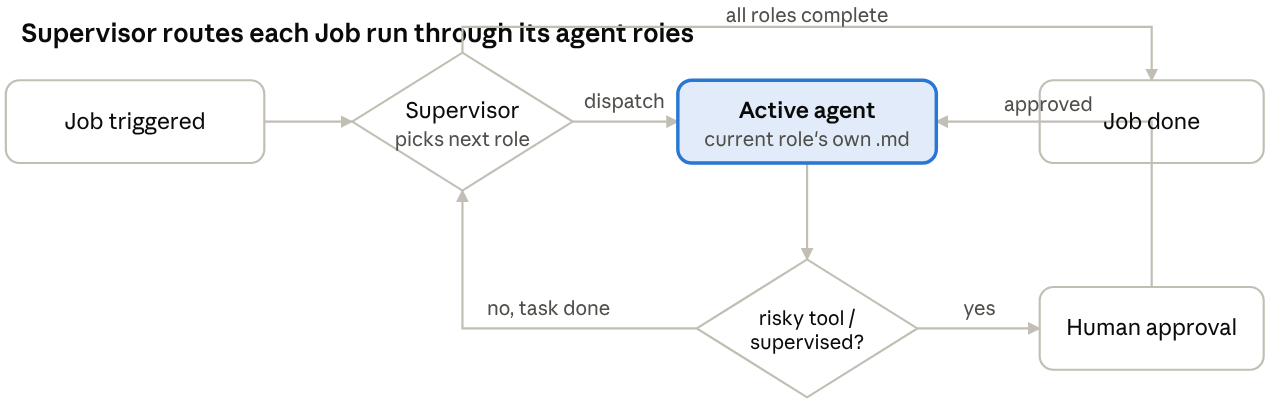
Level	What it is	Example
Job	The parent unit an end user triggers. Defines *which* agent roles participate, *how* they hand off to each other (the workflow graph), and *when* it runs (manual / scheduled / webhook).	"Regression QA Pass"
Agent	A role within a Job, defined entirely by a `.md` file (name, goal, instructions, tools, model, autonomy, who it reports to / hands off to). Reusable across Jobs.	QA Manager, QA Lead, Automation Engineer, Test Executor
Task	A unit of work scoped to one Agent inside one Job. Has instructions, inputs/outputs, and can depend on other Tasks (even on a different agent) before it can start.	"Break test plan into automatable vs manual cases" (owned by QA Lead)

A Job is therefore a **template** (agents + workflow + tasks) that can be **run** many times (manually or on a schedule); each execution is a **Run**, tracked exactly like today's `thread_id`, but now spanning multiple agents instead of one.

This directly answers your requirement: a Job can have multiple Agents (roles), and each Agent can own many Tasks driven by that role's own instructions -- nothing is hardcoded per-Job; the same QA Lead agent definition can be reused in a "Regression QA Pass" Job and a "Release Sign-off" Job with different tasks assigned to it.

2. Orchestration architecture

Today's `core/graph.py` compiles ONE planner/tool_executor/reflector loop. It is restructured into a **supervisor graph** that owns the Job's role sequence, plus one compiled sub-graph per Agent (the existing planner/tool_executor/reflector loop, reused as-is, but parameterized per agent by its `.md` definition -- system prompt, tools, model, autonomy).



Job run: supervisor dispatches each agent role in turn

- **Supervisor node** reads the Job's agent graph (an ordered or conditional list of roles, e.g. QA Manager -> QA Lead -> Automation Engineer -> Test Executor) and the current run's progress, then dispatches the next role's sub-graph.
- **Each agent sub-graph** is the existing planner -> tool_executor -> reflector loop, unchanged in shape, just built from that role's `.md` (prompt, tools, `risky_tools`, `autonomy_level`) instead of the single hardcoded `PLANNER_SYSTEM_PROMPT`.
- **human_approval** stays exactly as it works today (an `interrupt()`), now scoped per agent/task so the Log Explorer and resume calls can say *which* agent's *which* task is paused.
- When an agent finishes its task(s), control returns to the supervisor, which checks the Job's graph for the next role (static, admin-defined edges for v1; an optional LLM-decided dynamic router is a later phase) or ends the run.

- A Job's graph does not have to be a straight chain -- it supports fan-out (QA Manager assigns two parallel Automation Engineer instances) and conditional edges (skip Test Executor if Automation Engineer's coverage already meets a threshold), because it is just a small graph definition read by the supervisor, not new orchestration code per Job.

3. Agent definition format: .md, not Python

`core/prompts.py` (hardcoded `PLANNER_SYSTEM_PROMPT` / `REFLECTOR_SYSTEM_PROMPT` strings) is retired. Every agent role becomes one `.md` file under a new `agents/` folder: YAML frontmatter for everything structured, Markdown body for the instructions. A new `core/agent_loader.py` parses these into `AgentDefinition` objects at graph-build time -- no restart-and-redeploy just to change a role's wording.

```
---
name: qa_lead
display_name: QA Lead
reports_to: qa_manager
hands_off_to: qa_automation_engineer
model: claude-sonnet-5
llm_backend: claude-code
tools: [file_ops, test_plan_db, jira_api]
autonomy: checkpoint
risky_tools: [file_delete]
max_iterations: 15
max_tool_retries: 2
version: 1
---

## Role
You are the QA Lead. You receive an approved test strategy from the QA
Manager and turn it into an executable test plan.

## Instructions
1. Break the test plan into automatable vs. manual-only cases.
2. Assign automatable cases to the Automation Engineer as Tasks.
3. Assign manual-only cases to the Test Executor as Tasks.
4. Flag any case that is ambiguous back to the QA Manager instead of
guessing.
```

```
## Definition of done
- Every test case in the plan has exactly one owner (automation or manual).
- Each assigned Task states its acceptance criteria explicitly.
```

This is the exact shape your requirement describes: `reports_to` / `hands_off_to` is what the UI reads to draw the workflow diagram; `tools`, `autonomy`, `risky_tools` carry over the *existing* settings-level policy (today global in `core/config.py`) down to a per-agent level; `version` lets the registry keep history so an admin can diff or roll back a role's wording. A `_templates/base_agent_template.md` ships alongside for people writing one by hand.

4. Natural language -> structured .md pipeline

An end user (or advanced end user) should never have to hand-write YAML frontmatter. A new `core/nl_to_md.py` module adds a dedicated meta-agent (itself just another system prompt, run through the same `core/llm.py` backend already in the codebase) whose only job is converting plain English into a structured, schema-valid definition, for both Agents and Jobs:

1. User types a description in the UI ("I want an agent that reviews test plans and hands off automatable cases to an automation engineer").
2. The "Definition Writer" LLM call turns this into a draft `.md` (frontmatter + body), inferring `reports_to` / `hands_off_to` / a reasonable `tools` subset from context, and asks a follow-up question in the UI only if something required is missing (e.g. which existing role it hands off to).
3. The draft is validated against the Agent schema (`pydantic`) before it's ever shown -- a malformed draft is repaired automatically or flagged, never saved as-is.
4. The UI shows the generated `.md` side-by-side with the original plain-English request, editable inline, with a "Save" that writes the file to `agents/<name>.md` (or `jobs/<name>.yaml` for a Job) only on confirmation.
5. The same pipeline handles Job descriptions ("set up a nightly regression pass with a QA Manager, QA Lead, two Automation Engineers and a Test Executor") -- it proposes the *whole* role graph and default Tasks per role, which the user then edits on the visual canvas (Section 7) before saving.

This makes `.md` authoring accessible to a plain end user through natural language, while still giving an advanced end user (or an admin reviewing what was generated) the real structured file to edit directly -- nothing is hidden behind the generator.

5. Data model

`data/events.sqlite` (today: one flat `run_events` table) and `data/checkpoints.sqlite` (LangGraph's own checkpointer) both stay, but the events schema gains columns and several new tables arrive alongside. Recommend moving from SQLite to Postgres at this point (SQLite is fine for one user; a multi-user platform with concurrent runs needs real concurrency).

Table	Key columns	Purpose
`agents`	`name` (PK), `file_path`, `version`, `display_name`, `reports_to`, `hands_off_to`, `model`, `tools_json`, `autonomy`, `updated_at`, `updated_by`	Indexed mirror of `agents/*.md`, kept in sync by `agent_loader.py`; the UI reads this table, not the filesystem, for listing/searching
`jobs`	`id` (PK), `name`, `description`, `created_by`, `created_at`, `status` (`draft` / `published`), `trigger_type`	The parent Job definition
`job_agents`	`job_id`, `agent_name`, `position`	The workflow graph: which agents participate in this Job and how they're wired (edges come from each agent's own `reports_to` / `hands_off_to`, scoped to this Job)
`tasks`	`id` (PK), `job_id`, `agent_name`, `name`, `instructions`, `depends_on_json`, `status`	Work items owned by one agent inside one Job
`runs`	`id` (PK), `job_id`, `thread_id`, `status`, `started_at`, `finished_at`, `triggered_by` (`manual` / `schedule` / `webhook`), `triggered_by_user`	One execution of a Job -- the multi-agent equivalent of today's single `thread_id`

Table	Key columns	Purpose
`run_events` (extends existing)	*(existing columns)* + `level` (`DEBUG` / `INFO` / `WARN` / `ERROR`), `job_id` , `run_id` , `agent_name` , `task_id`	The log backbone -- this is the missing piece for the admin log requirement (today's table has no severity level)
`schedules`	`id` (PK), `job_id` , `cron_expr` , `enabled` , `last_run_at` , `next_run_at` , `created_by`	Persists what `interface/scheduler.py` 's `ScheduledAgent` currently only holds in memory
`users`	`id` (PK), `email` , `role` (`end_user` / `advanced_end_user` / `admin`), `created_at`	Auth + RBAC

6. API surface (extends interface/api.py)

Today's `/tasks*` , `/tools` , `/health` stay (backward compatible for single-agent runs); everything below is new.

Endpoint	Purpose
`GET/POST /agents`	List / create an Agent (`POST` body can be raw `.md` or `{nl: "..."}` for the NL pipeline, which returns a draft for confirmation before it writes anything)
`GET/PUT/DELETE /agents/{name}`	Read, update (new version), remove an Agent definition
`GET/POST /jobs`	List / create a Job (agents + workflow edges + tasks; NL pipeline supported the same way)
`GET/PUT/DELETE /jobs/{id}`	Full Job definition, including its graph
`GET /jobs/{id}/graph`	Nodes + edges for the workflow diagram (Section 7)
`POST /jobs/{id}/run`	Manually trigger a Run
`GET /jobs/{id}/runs`	Run history for a Job

Endpoint	Purpose
`GET /runs/{run_id}`	Status, current agent/task, per-agent sub-status
`GET /runs/{run_id}/events`	Filterable by `level`, `agent`, `task`, `module`, time range -- the admin Log Explorer's data source
`WS /runs/{run_id}/stream`	Live event + status stream (replaces polling)
`POST /runs/{run_id}/resume`	Existing approve/deny pattern, now scoped to the specific agent + task that's paused
`GET/POST/PUT/DELETE /schedules`	CRUD wired to `interface/scheduler.py`, persisted (not just in-memory)
`POST /schedules/{id}/run-now`	Fire a scheduled Job immediately
`POST /auth/login`, `GET /users/me`, `GET/POST /users` (admin only)	Auth + role management

7. UI/UX by role

End User

- **New Run**: pick a published Job, fill its inputs, submit -- or describe the task in plain English and let the NL pipeline propose a one-off Job for confirmation.
- **My Runs**: status list (queued/running/awaiting-approval/done/failed); clicking in shows the **workflow diagram** (Section 2's shape, rendered live) with each agent node colored by state -- pending / running / done / error / awaiting approval -- exactly answering "show how each role is linked in a Job, like the workflow."
- **Result view**: final output, per-agent intermediate outputs, downloadable artifacts.

Advanced End User (prompt-capable)

- **Agent Builder**: NL text box -> generated `.md` preview (editable) -> Save (Section 4's pipeline).
- **Job Builder**: a visual canvas (React Flow) -- drag existing Agents onto it, draw the `reports_to` / `hands_off_to` edges, or describe the whole workflow in plain English and let the pipeline draft the canvas, then adjust it by hand.

- **Task editor**: per agent node, add/edit Tasks (instructions, dependencies, inputs/outputs).
- **Library**: browse and fork existing Agents and Job templates rather than starting blank.

Admin

- **Flow Monitor**: every active Run across all Jobs, drill into one to see the live workflow diagram node-by-node (this is the same diagram component as the End User's Run view, with more detail per node: token usage, retries, timing).
- **Log Explorer** -- your original spec, now with the fields the new schema adds:
  - Timestamp (sortable, timezone-aware)
  - Module = `job_name / agent_name / task_id / node` (planner, tool_executor, reflector, human_approval)
  - Level: DEBUG / INFO / WARN / ERROR, filterable chips
  - Expandable JSON payload (tool inputs/outputs, LLM token counts)
  - Correlation by `run_id` (formerly `thread_id`) -- one click shows every log line for that Run across every agent
  - Search / regex, live tail via the new WebSocket endpoint
  - Export (CSV/JSON) for a date range or `run_id`
- **Scheduler**: create/edit cron schedules per Job, next-run countdown, run history, enable/disable, run-now.
- **Agent & Job registry**: approve/publish definitions an advanced end user authored, version diff/rollback per `.md`.
- **User & role management**, **system health** (queue depth, active runs, error rate, token usage per Job/Agent).

8. Tech stack (updated)

Layer	Choice	Note
Orchestration	LangGraph (kept)	Refactored to supervisor + per-agent subgraph
Agent definitions	Markdown + YAML frontmatter, `python-frontmatter` for parsing	Replaces `core/prompts.py` constants

Layer	Choice	Note
NL -> structured defs	Same `core/llm.py` backend (api or claude-code), new dedicated system prompt + Pydantic validation	No new LLM dependency
Backend API	FastAPI (kept), extended	+ WebSocket endpoint for live logs/status
Database	Postgres (upgrade from SQLite) via SQLAlchemy/SQLModel + Alembic migrations	Needed once multiple users/runs are concurrent
Scheduler	APScheduler (kept), wired into the API + persisted	`interface/scheduler.py` already exists, just needs endpoints + a DB-backed job store
Frontend	React + TypeScript	
Workflow diagram	React Flow	Renders the Job graph from `/jobs/{id}/graph`, live status overlay from the WebSocket
Data/tables	TanStack Query + TanStack Table	
Styling	Tailwind + shadcn/ui	
Charts	Recharts	Token usage, error rate, system health
Auth	JWT, role claims (`end_user` / `advanced_end_user` / `admin`)	

9. Repo structure changes

```
agents/                        NEW -- .md agent definitions (replaces
prompts.py)
  qa_manager.md
  qa_lead.md
  qa_automation_engineer.md
  qa_test_executor.md
  _templates/base_agent_template.md
```

```
jobs/                          NEW -- Job definitions (agents + workflow graph
+ tasks)
  qa_regression_job.yaml

core/
  agent_loader.py              NEW -- parses agents/*.md -> AgentDefinition
  job_loader.py                NEW -- parses jobs/*.yaml -> JobDefinition
  supervisor.py                NEW -- routes between agent sub-graphs per Job's
graph
  nl_to_md.py                  NEW -- NL -> structured .md/yaml (agents + jobs)
  graph.py                     CHANGED -- build_agent_subgraph(agent_def);
build_job_graph(job_def) assembles supervisor + subgraphs
  prompts.py                   CHANGED -- only the supervisor's and nl_to_md's
own meta-prompts remain here

interface/
  api.py                       CHANGED -- adds /jobs, /agents, /schedules,
/runs, WS stream
  scheduler.py                 CHANGED -- wired to API + DB-backed job store
(unchanged internals)

observability/
  tracer.py                    CHANGED -- adds level, job_id, run_id,
agent_name, task_id columns
```

10. Phased implementation roadmap

Phase	Scope
0 -- Foundations	Postgres schema (jobs/agents/tasks/runs/schedules/users), auth/RBAC
1 -- Agents as .md	`agent_loader.py`, migrate the 2 existing prompts to `.md`, `nl_to_md.py` for Agents, Agent CRUD API + Agent Builder UI
2 -- Multi-agent Jobs	`job_loader.py`, `supervisor.py` (rewrites `graph.py` into supervisor + per-agent subgraphs), Job CRUD + `/jobs/{id}/graph`, Job Builder UI (React Flow canvas)

Phase	Scope
3 -- Scheduling	Wire `interface/scheduler.py` into the API + DB-backed job store, Scheduler admin UI
4 -- Observability	Extend `tracer.py` schema (level/job/agent/task), WebSocket live stream, Log Explorer UI, Flow Monitor live diagram
5 -- Roles & polish	End User simple-run flow, Advanced End User NL Job builder, Admin user management, RBAC enforced on every endpoint

Recommended starting point: **Phase 1**, since every later phase (multi-agent orchestration, the workflow diagram, scheduling tied to Jobs) depends on Agents existing as loadable `.md` files rather than hardcoded prompts.